Parsek — Timeline System Design

Comprehensive design specification for Parsek's unified timeline view — a chronological, read-only query layer across recordings and game actions that gives the player a single place to see everything that has happened and will happen on the committed timeline.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to a timeline, rewind to earlier points, and see previously recorded missions play back as ghost vessels alongside new ones. This document specifies how the timeline aggregates and presents data from the flight recorder system (see `parsek-flight-recorder-design.md`) and the game actions system (see `parsek-game-actions-and-resources-recorder-design.md`).

Version: 1.0 (Implemented in v0.7) **Status:** Complete. **Out of scope:** Recording, playback, ghost visuals, vessel spawning. See `parsek-flight-recorder-design.md`. Resource tracking, ledger, recalculation engine. See `parsek-game-actions-and-resources-recorder-design.md`. Vessel-level telemetry (part events, segment events) — these remain in the Recordings Manager.

---

1. Introduction

Parsek has two existing views of committed data:

- The **Recordings Manager** is vessel-centric: a list of recordings with per-recording controls (enable/disable, rewind, loop, group). It answers "what vessels did I record?"
- The **Game Actions window** (replaced by the timeline) was resource-centric: a flat list of ledger actions and legacy game state events with sorting and deletion. It answered "what resource transactions happened?"

Neither answered the question the player actually has: **what happened, and what will happen, in chronological order across all systems?**

The timeline is a unified, chronological view that pulls from recordings and game actions, normalizes them into a common entry shape, and presents them sorted by universal time (UT). It replaced the Game Actions window entirely.

1.1 What the timeline is

The timeline is a **read-only query layer**. It does not own data. Recordings remain in `RecordingStore`. Game actions remain in the `Ledger`. The timeline reads from both, constructs a flat list of entries, and presents it.

This is analogous to a database view: it computes a result set from underlying tables. If a recording is committed or rewound, the timeline rebuilds. If a ledger action is added (facility upgrade, tech unlock), the timeline rebuilds. The timeline never modifies the data it reads.

1.2 What the timeline shows — and what it doesn't

The timeline shows **career-level events**: when missions launched and where vessels spawned, what milestones were achieved, what resources changed, when contracts completed, when the KSC was upgraded. These are the events that shape the player's career progression.

The timeline does **not** show vessel-level telemetry: engine ignitions, staging events, parachute deployments, solar panel toggles, RCS pulses. That detail belongs in the Recordings Manager. A decoupler firing or an engine throttle change is not a career event — it's a flight detail.

The dividing line: **if it changed a resource, a contract, a milestone, a kerbal, or a facility — it's in the timeline. If it only changed a vessel's physical state — it's in the Recordings Manager.**

1.3 What the player sees

The player sees the full committed history at all times. The timeline has a **current-UT marker** that divides the list into two visual halves:

- **Above the marker** (past): events that have already played out. Full color.
- **Below the marker** (future): events that will play out when the player advances time. Dimmed but not hidden.

There are no branches and no hidden future — the player recorded that future and committed it.

Situation	What the player sees in the timeline
Three recordings committed	All three recordings' career events interleaved chronologically — launches, spawns, milestones, science, contracts, all sorted by UT
Rewind to before a recording	That recording's events appear below the current-UT marker (future), dimmed.
Active flight in progress	Uncommitted events are not shown — they join the timeline on commit.
Facility upgrade at KSC	The upgrade action appears instantly in the timeline at the current UT.
Contract completed during a recording	The completion action appears at the UT it occurred, interleaved with other career events.

1.4 How the timeline differs from the Recordings Manager

Aspect	Recordings Manager	Timeline
Organized by	Vessel (one row per recording)	Time (one row per event)
Shows	Recording metadata, playback controls, chain/loop config, part events	Career events from all systems, interleaved chronologically
Controls	Enable/disable, rewind, loop, group, hide	Rewind/FF, filter, GoTo cross-link
Answers	"What recordings do I have? What did this vessel do?"	"What happened at UT 5000? How did my career progress?"

Both windows can be open simultaneously. The GoTo button on timeline entries opens the Recordings Manager and scrolls to the recording.

1.5 Data sources

1. Recordings (`RecordingStore.CommittedRecordings`)

Each committed recording contributes:

- A **launch event** at `rec.StartUT` with vessel name and MET duration. EVA recordings show `EVA: Kerbal from Vessel (MET 5s)` .
- A **spawn event** at `rec.EndUT` with vessel situation context: `Spawn: Vessel (Landed on Mun)` . Boarded EVA kerbals show `Board: Kerbal (Vessel)` .
- Debris recordings and hidden recordings are skipped.

2. Game actions (`Ledger.Actions`)

Each ledger action is a single timeline entry. Display text is humanized: science subjects split and spaced (`Crew Report @ Kerbin Launchpad`), tech nodes capitalized (`Basic Rocketry`), milestones with spaces (`First Launch`), strategy names from lookup table (`Aggressive Negotiations`). Crew assignments include vessel name (`Assign: Jeb (Pilot) on Mun Lander`). EVA self-assignments (kerbal assigned to own EVA vessel) are filtered out.

Game actions are classified as either **Actions** (deliberate player choices: tech unlock, build, hire, contract accept/cancel, facility upgrade/repair, strategies) or **Events** (gameplay consequences: milestones, science earned, contract complete/fail, reputation changes, crew assignment/rescue).

3. Legacy events (`MilestoneStore.Milestones`)

Saves started before the ledger system have committed events stored as `GameStateEvent` entries in `MilestoneStore` . These disappear as saves migrate to the ledger system. Uncommitted events are **not** shown.

---

2. Design Philosophy

2.1 Read-only, not read-write

The timeline displays data. It never modifies it. Rewind and fast-forward are operations on the game state (loading a quicksave, advancing UT), not on the timeline's data.

2.2 Time-centric, not vessel-centric or resource-centric

The timeline groups by **time**. This reveals causal chains:

> *UT 600: Milestone: First Orbit +5000 funds UT 800: Upgrade Launch Pad → Lv.2 -18000 UT 900: Complete: Orbit Kerbin +8000 funds*

The player sees that the milestone funded the runway upgrade.

2.3 Career events, not vessel telemetry

The timeline shows things that change career state. It does not show what a vessel did physically — that's the Recordings Manager's job.

2.4 Full visibility, no hidden future

Both past and future events are always visible. Future events are dimmed but never hidden.

2.5 Significance filtering

Two tiers: **Overview** (mission structure) and **Detail** (all transactions). Default is Overview.

2.6 The timeline replaces the Game Actions window

The timeline lives in `UI/TimelineWindowUI.cs` , replacing `UI/ActionsWindowUI.cs` . Same extracted-window pattern as the other UI classes: constructor takes `ParsekUI` parent, `DrawIfOpen(Rect)` entry point, `IsOpen` property, `ReleaseInputLock()` .

---

3. Architecture

3.1 Coupling to other systems

```
RecordingStore ──reads──→ TimelineBuilder ←──reads── Ledger
                              ↑
                          reads│ (legacy saves only)
                              │
                         MilestoneStore
```

The only side effect is through rewind/FF buttons, which delegate to `RecordingsTableUI.ShowRewindConfirmation()` and `ShowFastForwardConfirmation()`.

3.2 TimelineEntry — the normalized event shape

```
TimelineEntry
    double            UT              — universal time
    TimelineEntryType Type            — discriminator enum (see §3.3)
    string            DisplayText     — one-line human-readable description
    TimelineSource    Source          — Recording, GameAction, or Legacy
    SignificanceTier  Tier            — T1 or T2
    Color             DisplayColor    — green (earning), red (spending), light blue (player
action), white (neutral)
    string            RecordingId     — source recording ID; null for KSC-only actions
    string            VesselName      — vessel name if applicable; null otherwise
    bool              IsEffective     — false if zeroed by recalculation (duplicate milestone,
etc.)
    bool              IsPlayerAction  — true = deliberate KSC action, false = gameplay event
```

`TimelineEntry` is a **view object** — constructed on demand, never serialized, never persisted.

3.3 TimelineEntryType

Recording lifecycle (2 types): `RecordingStart`, `VesselSpawn`

Game actions (23 types, 1:1 with `GameActionType`): `ScienceEarning`, `ScienceSpending`, `FundsEarning`, `FundsSpending`, `ReputationEarning`, `ReputationPenalty`, `MilestoneAchievement`, `ContractAccept`, `ContractComplete`, `ContractFail`, `ContractCancel`, `KerbalAssignment`, `KerbalHire`, `KerbalRescue`, `KerbalStandIn`, `FacilityUpgrade`, `FacilityDestruction`, `FacilityRepair`, `StrategyActivate`, `StrategyDeactivate`, `FundsInitial`, `ScienceInitial`, `ReputationInitial`

Legacy (1 type): `LegacyEvent`

Total: 26 types.

3.4 Entry collection

`TimelineBuilder.Build()` accepts data sources as parameters for testability:

```
TimelineBuilder.Build(
    IReadOnlyList<Recording> committedRecordings,
    IReadOnlyList<GameAction> ledgerActions,
    IReadOnlyList<Milestone> milestones,
```

```
    uint currentEpoch
) → List<TimelineEntry>
```

Three collectors then stable sort by UT:

Recording Collector — emits `RecordingStart` (with MET duration, EVA detection, parent vessel resolution) and `VesselSpawn` at EndUT (with terminal state and VesselSituation). Skips hidden and debris recordings. Chain recordings show full chain duration. EVA detection via `EvaCrewName` or single-crew vessel name match.

Game Action Collector — maps types 1:1, humanizes display text (science subjects, tech nodes, milestones, strategies, crew assignments with vessel name), classifies as Action or Event via `IsPlayerAction`, demotes ineffective T1 entries to T2, resolves vessel name from RecordingId.

Legacy Collector — iterates committed milestones matching current epoch, skips filtered event types, all entries at T2.

3.5 Cache invalidation

Rebuilt on: recording commit, rewind, KSC spending, scene change, warp exit. Current-UT divider updates every draw call via `Planetarium.GetUniversalTime()`.

3.6 Display text humanization

- **Science subjects**: `crewReport@KerbinSrfLaunchpad` → `Crew Report @ Kerbin Launchpad` (camelCase split, `Srf Landed` / `Srf Splashed` /standalone `Srf` stripped)
- **Tech nodes**: `basicRocketry` → `Basic Rocketry`
- **Milestones**: `FirstLaunch` → `First Launch`, `/` → `-` (body separator)
- **Strategies**: lookup table for 7 stock strategies (`AggressiveNeg` → `Aggressive Negotiations`), camelCase fallback for mods
- **Duration**: `FormatDuration` with KSP calendar (6h days, 426d years), only non-zero components

---

4. Significance Tiers

4.1 T1 — Overview (default)

Entry Type	Why T1
RecordingStart	Mission launch — fundamental timeline anchor
VesselSpawn	Vessel materialization with terminal state — the outcome
MilestoneAchievement	One-time progression gate, often with large rewards
ContractComplete	Mission objective achieved
ContractFail	Mission objective failed
FacilityUpgrade	KSC progression
FacilityDestruction	KSC regression
KerbalHire	Roster expansion
FundsInitial	Career start — baseline reference

ScienceInitial	Career start
ReputationInitial	Career start

4.2 T2 — Detail

All remaining entry types: resource transactions (science/funds/rep earning and spending), contract accept/cancel, crew assignment/rescue/stand-in, facility repair, strategy activate/deactivate, legacy events.

4.3 Visibility rules

- **Overview** (default): T1 only.
- **Detail**: T1 + T2.
- **Ineffective entries**: demoted one tier (T1→T2). Rendered with strikethrough and gray.

---

5. Player Interaction

5.1 Window layout

Four zones top to bottom:

Zone 1: Resource Budget — reserved vs. available funds/science/reputation. Hidden in sandbox, science-only in science mode.

Zone 2: Filter Bar — tier selector (Overview / Detail), source toggles (Recordings / Actions / Events). All toggle states reflected in footer counts.

Zone 3: Entry List — flat chronological list with current-UT divider. Past entries in full color (green = earnings, red = penalties, light blue = player actions, white = recordings). Future entries dimmed. Each RecordingStart entry has R (past) or FF (future) button and GoTo button.

Zone 4: Footer — visible entry counts adapting to active filters ("5 Recordings, 8 Actions, 15 Events"), retired kerbals section, close button.

5.2 Entry display examples

```
Launch: Kerbal X (MET 6m, 30s)                    — regular recording
EVA: Jeb from Kerbal X (MET 5s)                    — EVA recording
Spawn: Kerbal X (Landed on Mun)                    — vessel materializes at EndUT
Board: Jeb (Kerbal X)                              — EVA kerbal reboarded
Milestone: First Launch +5000 funds +2.5 rep        — humanized milestone
Complete: Orbit Kerbin +8000 funds                  — contract
Tech: Basic Rocketry -5.0 sci                       — humanized tech node
Crew Report @ Kerbin Launchpad +5.0 sci             — humanized science
Build -5000                                         — vessel build cost
Assign: Jeb (Pilot) on Kerbal X                    — crew with vessel
Activate: Aggressive Negotiations (25% Funds→Rep)   — full strategy name
Starting funds: 25000                               — career seed
```

5.3 Rewind / Fast-Forward

RecordingStart entries show R (past recordings with rewind save) or FF (future recordings). Both delegate to `RecordingsTableUI` methods — same confirmation dialogs and execution as the Recordings Manager.

5.4 GoTo cross-link

GoTo button on RecordingStart entries opens the Recordings Manager, unhides the recording if hidden, expands all parent groups in the hierarchy, and scrolls to the recording via deferred rendered-row detection.

5.5 Ineffective entries

Demoted one tier (T1→T2). Rendered with strikethrough text and gray color.

---

6. Game Mode Considerations

- **Career**: all features active.
- **Science mode**: budget shows only science. Fund/reputation/contract/strategy entries never appear.
- **Sandbox**: budget hidden. Only recording lifecycle events shown.

---

7. What the Timeline Does Not Do

- **Vessel-level telemetry**: part events, segment events, flag events — Recordings Manager only.
- **Resource sparkline / graphs**: deferred.
- **Individual event deletion**: read-only.
- **Sorting by columns**: always UT order.
- **Live UT marker during warp**: jumps on warp exit.
- **Uncommitted events**: not shown.